Prof. Björn Gambäck
TA. Tollef Jørgensen

Lab 1 – Corpora and Text Processing

Jonathan Brooks, jonatbr

Instructions

To run the code you only need the packages specified in the practice lecture, and maybe the json package which you can find on pip.

Exercises

1 - Managing text and filtering

A - Print all words beginning with *fo*

```
words = ["further", "Forward", "Foreign", "financE", "Forgive", "feature",
    "federal", "failurE", "Feeling", "finding ", "freedom", "Foundry" ]

def allWordsBeginningWithFo(list):
    for word in list:
        if re.search("^fo", word, re.IGNORECASE):
            print(word)

allWordsBeginningWithFo(words)
```

When running the code you get the following output.

```
Forward
Foreign
Forgive
Foundry
```

B - Print all words ending with *e*

Using the same words list as before.

```
def allWordsEndingWithE(list):
    for word in list:
        if re.search(".*e$", word, re.IGNORECASE):
            print(word)

allWordsEndingWithE(words)
```

When running the code you get the following output.

```
financE
Forgive
feature
failurE
```

C - Reflect on how simple techniques like these can bring value to a project

Techniques like this can be very useful to for example filter out words that do not add any meaning to a sentence, so called stopwords. Stopwords can be words like *is* or *you*. We want to filter out these types of words to have less data to analyse.

D - Converting words to lowercase is a frequent process in text cleaning. Can you think of any issues that arise by doing this?

When converting words to lowercase they can lose some of their meaning, words such as proper nouns. The capitalised first letter denotes that the words represents for example someones name or a company name.

2 - Reading a corpus

A - Look at the top 5 most common words. Do you notice any similarities? Explain your findings

```
import nltk
from nltk.corpus import brown

categories = ["government", "news", "humor"]

governmentWords = brown.words(categories=categories[0])
newsWords = brown.words(categories=categories[1])
humorWords = brown.words(categories=categories[2])

governmentFD = nltk.FreqDist(governmentWords)
newsFD = nltk.FreqDist(newsWords)
humorFD = nltk.FreqDist(humorWords)

print("--- Government top 5 words ---")
print(governmentFD.most_common (5))
print("--- News top 5 words ---")
print(newsFD.most_common (5))
print("--- Humor top 5 words ---")
print(humorFD.most_common (5))
```

Which gives:

```
--- Government top 5 words ---
[('the', 4143), (',', 3405), ('of', 3031), ('.', 2493), ('and', 1923)]
--- News top 5 words ---
[('the', 5580), (',', 5188), ('.', 4030), ('of', 2849), ('and', 2146)]
--- Humor top 5 words ---
[(',', 1331), ('the', 930), ('.', 877), ('of', 515), ('and', 512)]
```

As the data shows, all of the three categories contain the same 5 words and punctuation's. The *real* words are some of the most used words in the English language, **the** is actually the most used. This is why we can expect to see them. There is also always at least one punctuation in a sentence, that's why we also see them in the top 5.

B - Filter *stopwords* and repeat A. Describe at least two new techniques you would use to further improve the result

```
import nltk
from nltk.corpus import brown, stopwords

categories = ["government", "news", "humor"]

stops = stopwords.words()

governmentWords = [x for x in brown.words(categories=categories[0])
                     if x not in stops]
newsWords = [x for x in brown.words(categories=categories[1])
```

```
                if x not in stops]
humorWords = [x for x in brown.words(categories=categories[2])
                if x not in stops]

governmentFD = nltk.FreqDist(governmentWords)
newsFD = nltk.FreqDist(newsWords)
humorFD = nltk.FreqDist(humorWords)

print("--- Government top 5 words ---")
print(governmentFD.most_common(5))

print("--- News top 5 words ---")
print(newsFD.most_common(5))

print("--- Humor top 5 words ---")
print(humorFD.most_common(5))
```

Which gives:

```
--- Government top 5 words ---
[(',', 3405), ('.', 2493), ('The', 478), (';', 444), (')', 345)]
--- News top 5 words ---
[(',', 5188), ('.', 4030), ('The', 806), ('``', 732), ("''", 702)]
--- Humor top 5 words ---
[(',', 1331), ('.', 877), ('``', 343), ("''", 340), ('I', 239)]
```

The given data does not really give more information about what the data represents, therefore I need some more techniques to filter out the bad data. One technique I would do is to make sure that both the word and the stopword that it is being compared too are using the same case, to make sure we get out all stopwords. Also, I would remove all special characters from the corpus.

C - Implement at least one technique of your choice and repeat A

I chose to remove special characters, and turn the word being compared to the stopwords into lower case. Which gives us data that represents the corpus a lot better.

```
import nltk
from nltk.corpus import brown, stopwords

categories = ["government", "news", "humor"]

stops = stopwords.words()

governmentWords = [x for x in brown.words(categories=categories[0])
                     if x.lower() not in stops and x.isalnum()]
newsWords = [x for x in brown.words(categories=categories[1])
                if x.lower() not in stops and x.isalnum()]
humorWords = [x for x in brown.words(categories=categories[2])
                if x.lower() not in stops and x.isalnum()]

governmentFD = nltk.FreqDist(governmentWords)
newsFD = nltk.FreqDist(newsWords)
humorFD = nltk.FreqDist(humorWords)

print("--- Government top 5 words ---")
print(governmentFD.most_common(5))

print("--- News top 5 words ---")
print(newsFD.most_common(5))

print("--- Humor top 5 words ---")
print(humorFD.most_common(5))
```

```
--- Government top 5 words ---
[('year', 183), ('States', 162), ('United', 155), ('may', 153), ('would', 120)]
--- News top 5 words ---
[('said', 402), ('would', 244), ('last', 161), ('two', 157), ('new', 148)]
--- Humor top 5 words ---
[('said', 87), ('would', 56), ('time', 43), ('even', 38), ('like', 34)]
```

D - Plot a feature you find interesting

I chose to plot the dispersion of the 5 most common words in the brown corpus, from the news category.

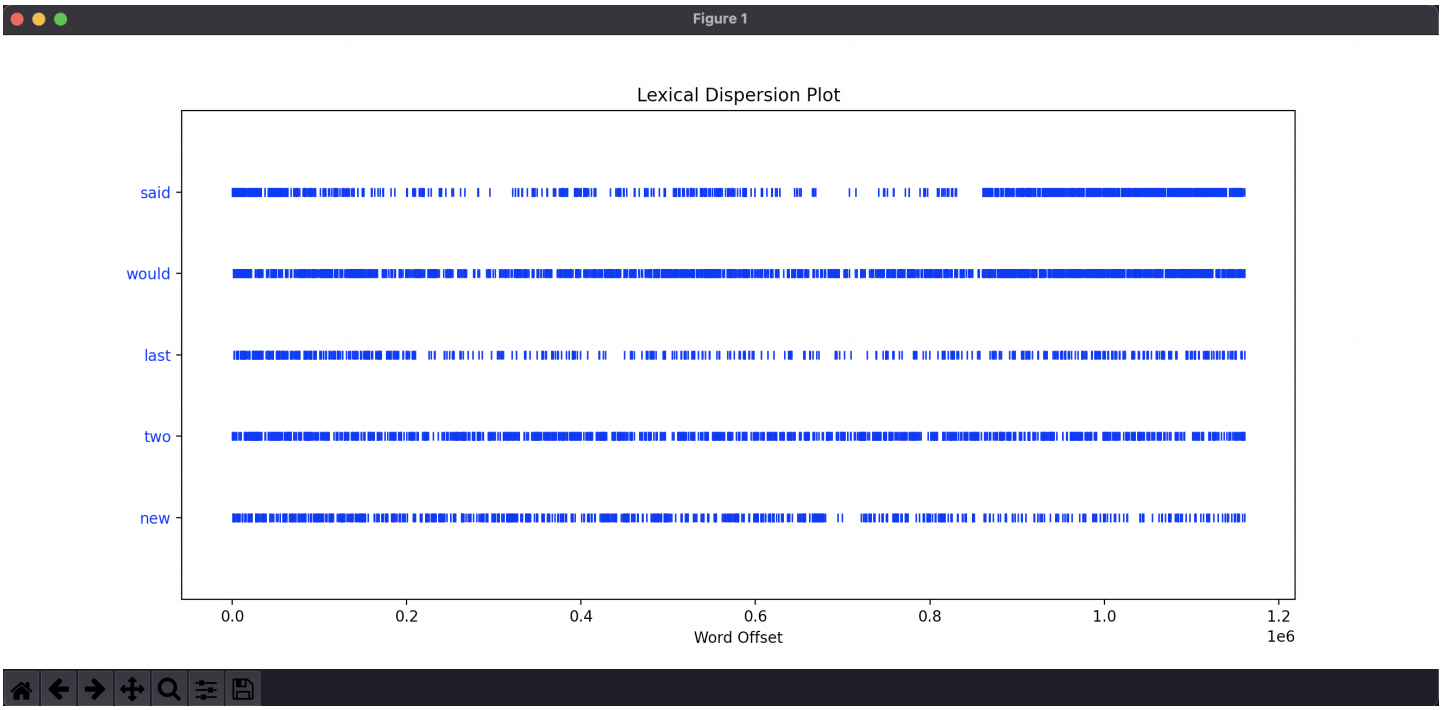


Figure 1: Dispersion

```python
import nltk
from nltk.corpus import brown, stopwords
from nltk.text import Text

stops = stopwords.words()

newsWords = [x for x in brown.words(categories="news")
                if x.lower() not in stops and x.isalnum()]

newsFD = nltk.FreqDist(newsWords).most_common(5)

Text(brown.words()).dispersion_plot([x[0] for x in newsFD])
```

3 - Building your own corpus

A - Build the corpus

I chose to read articles from E24. Code should be attached.

B - Clean the data using what you have just learned

I cleaned away the Norwegian stopwords and removed punctuation's.

A problem I stumbled upon is that when I split up my words by space, some words will still include punctuation's. Then when I check if words are alnum I remove these words. So I should check if words contain punctuation's, and if they do remove them.

C - Separate the text body (sentences) into tokens (words) by splitting on space

As mentioned in B, an improvement I want to make is to remove the punctuation's from the words instead of removing all words with punctuation's. So this is the improvement I made. Also some places there are double spaces, so I remove the empty words from the corpus.

D - Top 10 most common words

```
[('Oslo', 335), ('prosent', 310), ('ser', 310), ('sier', 286), ('kroner', 227),
   ('E24', 215), ('levert', 175), ('Schibsted', 170), ('Kurser', 168), ('ICE', 168)]
```

E - Top 10 most common bigrams

```
[(('ser', 'annonsen'), 168), (('Oslo', 'Bors'), 137), (('Bors', 'Oslo'), 88),
   (('lese', 'Flere'), 84), (('Flere', 'artikler'), 84), (('arbeider', 'Vaer'), 84),
   (('Vaer', 'varsomplakatens'), 84), (('varsomplakatens', 'regler'), 84),
   (('regler', 'presseskikk'), 84), (('presseskikk', 'Redaktorplakaten'), 84)]
```

Note: I had to edit out non-ascii characters, like æ and ø, to display them in Latex.
I see now that there are a lot of words that are present on every E24 article that do not add any value to the article, and some of them should probably have been filtered out.

F - Headline with most frequent words

```
('Hydrogenpro kraftig opp paa positiv Oslo Bors', 2)
```

Note: I had to, again, edit out non-ascii characters, like æ and ø, to display them in Latex.